МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний університет «Полтавська політехніка імені Юрія Кондратюка»

Кафедра комп’ютерних та інформаційних технологій і систем

Звіт

з лабораторної роботи №2

з дисципліни «Технології розробки корпоративних Web-додатків»,

тема: «Проєктування архітектури програмного забезпечення та моделювання даних»

Перевірив:

ас. каф. КІТС Мізік Ф.О.

Виконала:

студентка групи 404-ТН

Кривонос Юлія

Полтава, 2026

Розділ 1. Обґрунтування вибору технологічного стеку

Для реалізації мобільного вебдодатка «Moon» було обрано сучасний стек технологій, що дозволяє швидко розробляти масштабовані рішення з акцентом на UX/UI та реальну синхронізацію даних.

Визначення технологій:

- Мова програмування та Back-end: Firebase (BaaS — Backend as a Service). Замість класичного серверного фреймворку (Node.js чи .NET) використовується хмарна платформа від Google, яка надає готові сервіси для авторизації та роботи з базою даних через SDK.

- Фреймворк Front-end: React.js (Vite). Бібліотека для створення динамічних інтерфейсів на основі компонентів.

- Система керування базами даних (СКБД): Cloud Firestore (NoSQL). Гнучка документна база даних від Firebase, що підтримує синхронізацію в реальному часі.

- Інструменти DevOps/Hosting: GitHub Actions (для автоматизації деплою) та Firebase Hosting (для швидкого та безпечного розміщення додатка в мережі).

Обґрунтування вибору:

- React + Vite: Обрано через високу швидкість розробки та величезну кількість готових бібліотек для створення мобільних інтерфейсів. Vite забезпечує миттєве перезавантаження сторінки під час розробки, що критично важливо для ітеративного дизайну.

- Firebase Authentication: Це рішення дозволяє реалізувати безпечний вхід через Google Auth (як було зроблено в додатку) за лічені хвилини, не створюючи власну складну систему збереження паролів.

- Cloud Firestore (NoSQL): Оскільки структура даних фільмів може бути гнучкою (різні фільми можуть мати різні набори метаданих, як-от рейтинги, посилання на трейлери чи описи), NoSQL модель документів ідеально підходить для цього проєкту. ТакожFirestore підтримує offline-режим, що є обов'язковим для стратегії PWA.

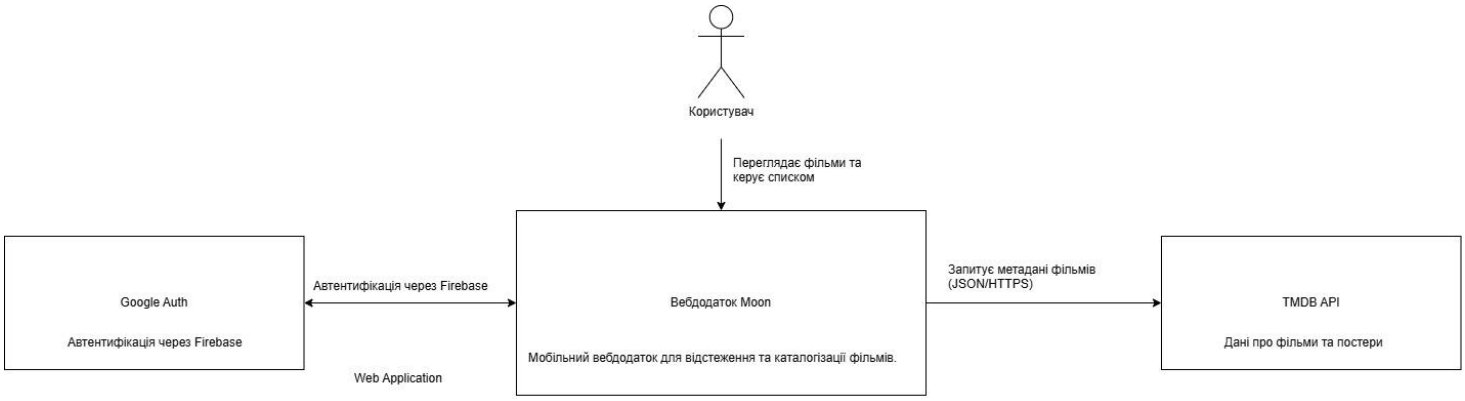
- BaaS підхід (Serverless): Використання Firebase замість написання власного API на C# чи Python дозволяє розробнику зосередитися на фронтенд-частині та дизайні, значно скорочуючи час виходу продукту

на ринок (Time-to-Market).

- External API (TMDB): Для автоматичного наповнення каталогу обрано TMDB API, що позбавляє необхідності вручну створювати величезну базу даних фільмів та постерів.

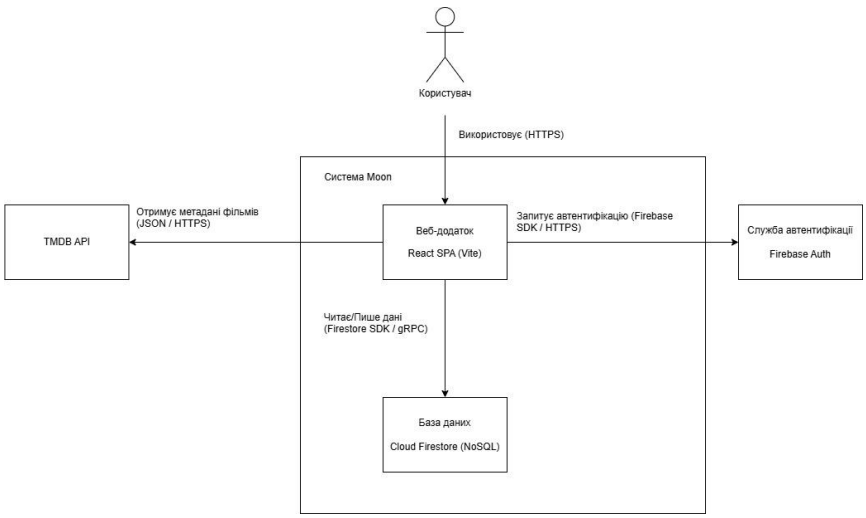
Розділ 2. Архітектурне моделювання за методологією C4 (C4 Model)

Рівень 1: System Context Diagram (Контекст)



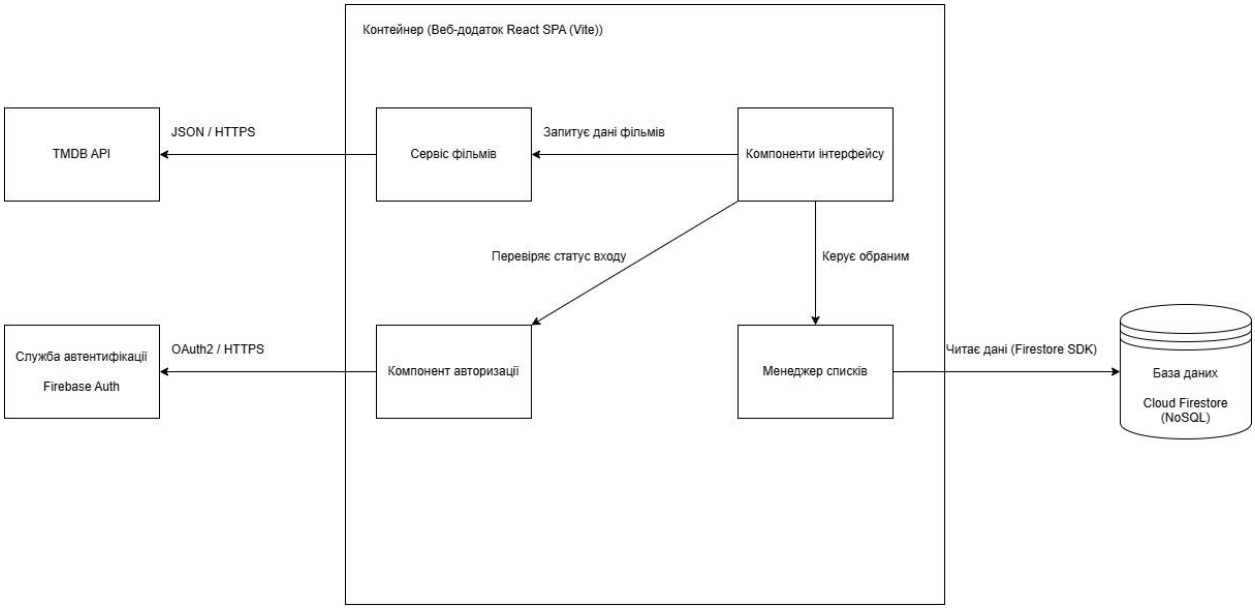
Діаграма контексту (Level 1) демонструє загальну архітектуру системи "Moon" та її взаємодію із зовнішнім середовищем. Головним вузлом є сам мобільний вебдодаток, з яким безпосередньо взаємодіє Користувач для перегляду та каталогізації фільмів. Для забезпечення повноцінного функціоналу система інтегрується з двома зовнішніми сервісами: сервісом Google Auth (через Firebase) для безпечної автентифікації та зовнішнім інтерфейсом TMDB API для отримання актуальних метаданих про фільми (постери, описи) за допомогою JSON/HTTPS запитів.

Рівень 2: Container Diagram (Контейнери)



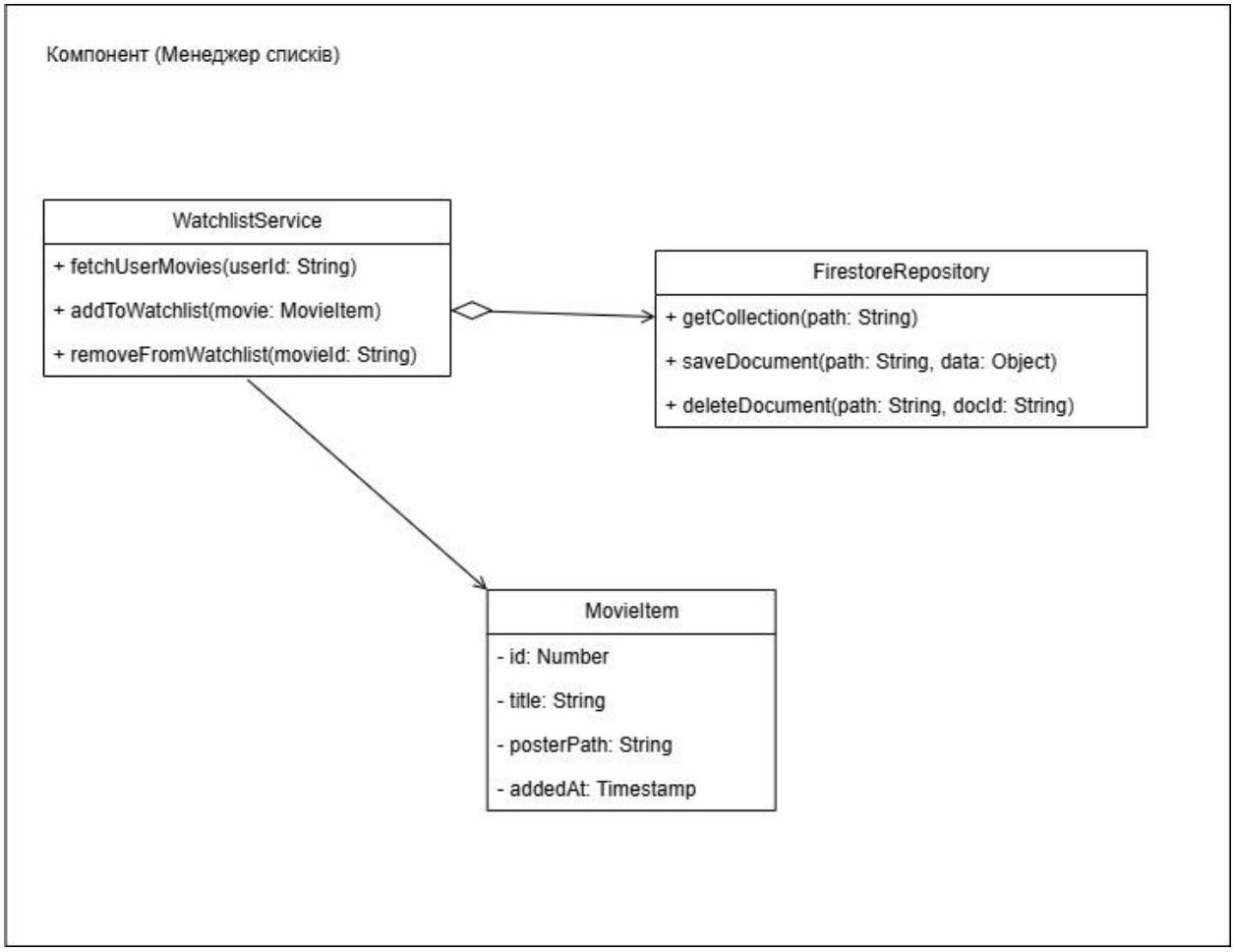
Діаграма контейнерів (Level 2) розкриває базову архітектуру системи "Moon", деталізуючи її на окремі логічні блоки. Центральним вузлом є клієнтський Вебдодаток (React SPA на базі Vite), який обробляє всі взаємодії з Користувачем. Для збереження користувацьких даних та списків фільмів додаток напряму взаємодіє з нереляційною Базою даних Cloud Firestore (через Firestore SDK / gRPC). Також продемонстровано інтеграцію із зовнішніми системами: службою Firebase Auth для безпечного управління автентифікацією та TMDB API для завантаження актуальних метаданих кінофільмів (через JSON/HTTPS запити).

Рівень 3: Component Diagram (Компоненти).



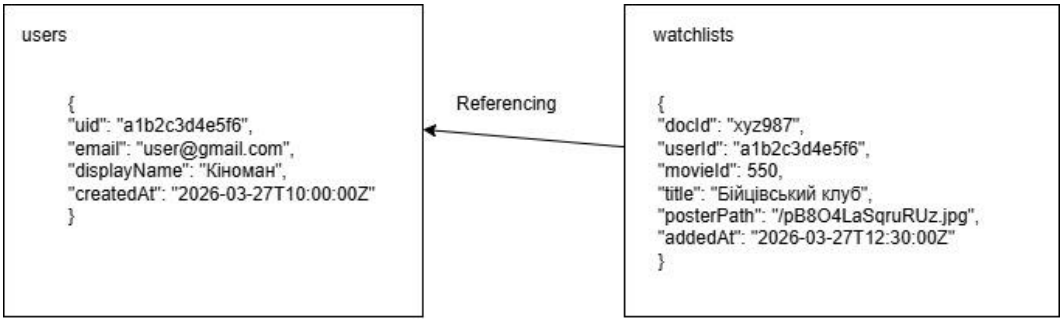
Діаграма компонентів (Level 3) деталізує внутрішню архітектуру клієнтського вебдодатка "Moon", розділяючи його на ключові функціональні модулі. Центральним елементом виступають Компоненти інтерфейсу, які оркеструють роботу системи та делегують задачі спеціалізованим сервісам. Зокрема, Сервіс фільмів відповідає за комунікацію із зовнішнім TMDB API (отримання даних через JSON/HTTPS), Менеджер списків забезпечує читання та збереження обраних фільмів у базі Cloud Firestore (через Firestore SDK), а Компонент авторизації здійснює перевірку статусу входу, взаємодіючи зі службою Firebase Auth (за протоколом OAuth2/HTTPS).

Рівень 4: Code Diagram (Код).



Діаграма коду (Level 4) ілюструє внутрішню об'єктну структуру компонента "Менеджер списків". Головним класом виступає WatchlistService, який інкапсулює бізнес-логіку керування обраними фільмами користувача. Через відношення агрегації він взаємодіє з класом FirestoreRepository, делегуючи йому виконання базових операцій із хмарною базою даних (отримання, збереження та видалення документів). Крім того, сервіс пов'язаний спрямованою асоціацією з моделлю даних MovieItem, що забезпечує чітку типізацію структури документа фільму (ідентифікатор, назва, шлях до постеру) для безпечної та передбачуваної обробки інформації.

Розділ 3. Проєктування бази даних (ER-Model / Data Schema)



Стратегія зв'язків та зберігання даних (NoSQL)

Для бази даних Cloud Firestore обрано гібридну стратегію.

Referencing (Посилання на ID): використовується для зв'язку між користувачем та його списком фільмів. У документах колекції watchlists зберігається поле userId. Документ користувача в Firestore має ліміт розміру (1 МБ). Оскільки список улюблених фільмів може нескінченно зростати, Referencing дозволяє уникнути переповнення документа користувача і забезпечує швидкий пошук списку за допомогою індексів (where("userId", "==", uid)).

Embedding (Вкладення даних): використовується для зберігання метаданих фільму (поля title, posterPath) безпосередньо в документі watchlists, хоча ці дані належать зовнішньому TMDB API. Це дозволяє миттєво відображати список збережених фільмів в інтерфейсі за один запит до БД, уникаючи необхідності робити десятки додаткових HTTPS-запитів до TMDB API для отримання постерів та назв кожного окремого фільму зі списку.